University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

Operating System

Lab No 4

Process Creation using fork()

Faculty of Information Technology

UCP Lahore Pakistan

Muhammad Ahmar Khan, Department of Software Engineering

University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

Process Creation Using Fork

Topic to be covered

- *Process Creation*
  - fork ()
  - getpid()
  - getppid()
- *Process Completion*
  - wait (int *)
  - exit (int)
- *Orphan Process*
- Zombie Process
- Multiple forks in a program.
- Example Programs for proof of concept

Objectives

- Understanding the concept of creating new processes in Linux.
- Understanding the process id and parent process id using getpid() and getpid()
- Parent waiting for its child to terminate using wait() system call.
- Concept of orphan and zombie Processes.
- Understanding the use of multiple forks in a program using tree diagram.

Prerequisite:

- Visual Studio Code
- GCC compiler
- Basic C Programing
- Use of man Page

University of Central Punjab

(Incorporated by Ordinance No. XXIV of 2002 promulgated by Government of the Punjab)

FACULTY OF INFORMATION TECHNOLOGY

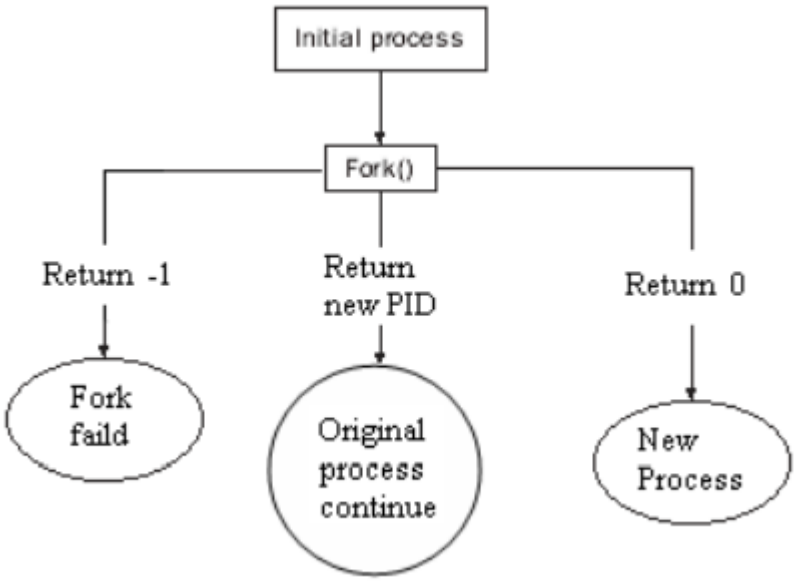
Process Creation in Linux:

In an operating system, the `fork()` system call is used to create a new process, which is essentially a duplicate of the calling process. This new process is called the child process, while the original is the parent process. The `fork()` function does not take any parameters and returns a `pid_t` type, representing the process ID.

After `fork()` is called, two processes will run concurrently. Both the parent and child processes will execute the next instruction following the `fork()`. To differentiate between them, the return value of `fork()` must be checked:

- If `fork()` returns a negative value, the creation of the child process failed.
- A return value of zero indicates that the current process is the child.
- A positive value indicates that the current process is the parent, and the returned value is the process ID of the child.

The function `getpid()` can be used by a process to obtain its own process ID, which helps in identifying the process uniquely. This mechanism is fundamental for multitasking in operating systems, allowing processes to perform independent tasks.



pid_t fork()

Simple Process Creation

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("I am the child process.\n");
    } else {
        printf("I am the parent process.\n");
    }

    return 0;
}
```

Task 1: Run the program, observe the output, and explain it.

Fork with Error Handling

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0) {
        printf("Child process.\n");
    }
    else {
        printf("Parent process.\n");
    }

    return 0;
}
```

Task 2: Run the program, observe the output, and explain it.

Understanding Process IDs

Every Linux process has a unique process ID (PID) that helps in identifying it. We can use `getpid()` to retrieve the PID of the current process and `getppid()` to get the PID of the parent process.

Displaying Process IDs

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid == -1) {
        printf("Fork failed!\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process. PID: %d Parent PID: %d\n",
               getpid(), getppid());
    }
    else {
        printf("Parent Process. PID: %d\n", getpid());
    }

    return 0;
}
```

Task 3: Run the program, observe the output, and explain it.

Example 4: Check Process ID Before and After Fork

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Before fork. Current process PID: %d\n", getpid());

    pid_t pid = fork();

    if (pid == 0) {
        // Child process
        printf("After fork in child. Child PID: %d, Parent PID: %d\n",
               getpid(), getppid());
    }
    else {
        // Parent process
        printf("After fork in parent. Parent PID: %d\n", getpid());
    }

    return 0;
}
```

Task 4: Run the program, observe the output, and explain it.

Waiting for Child Process

The `wait()` system call makes the parent process wait until the child finishes its execution, ensuring synchronized execution.

Parent Waiting for Child Process

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid = fork();

    if (pid == -1) {
        printf("Fork failed!\n");
```

```
    return 1;
  }

  if (pid == 0) {
    printf("Child Process. PID: %d\n", getpid());
    exit(0);
  }
  else {
    wait(NULL);
    printf("Parent Process. PID: %d\n", getpid());
  }

  return 0;
}
```

Task 5: Run the program, observe the output, and explain it.

Process Completion:

The functions described in this section are used to **wait** for a child process to terminate or stop, and determine its status. These functions are declared in the header file "**sys/wait.h**".

pid_t wait (int *status)

wait() will force a parent process to wait for a child process to stop or terminate. **wait()** return the pid of the child or

-1 for an error. The exit status of the child is returned to **status**.

void exit (int status)

exit() terminates the process which calls this function and returns the exit status value. Both UNIX and C (forked) programs can read the status value.

Parent Waiting for Child to Terminate

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>

int main() {
    pid_t cpid = fork();
    if (cpid == -1) {
        printf("Fork failed\n");
        exit(1);
    }
    if (cpid == 0) {
        // Child process
        printf("Hello, I am the Child, and my PID is: %d\n", getpid());
        exit(0);
    }
    else {
        // Parent process
        wait(NULL);
        printf("Hello, I am the Parent, and my PID is: %d\n", getpid());
    }
    return 0;
}
```

Task 6: Run the program, observe the output, and explain it.

Orphan processes:

When a parent dies before its child, the child is automatically adopted by the original "init" process whose **PID** is 1. To illustrate this insert a **sleep** statement into the child's code. This ensured that the parent process terminated before its child.

Creating an Orphan Process

```
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    pid_t pid;

    printf("I am the original process with PID %d and PPID %d\n",
           getpid(), getppid());

    pid = fork();

    if (pid > 0) {
        // Parent process
        printf("I am the parent with PID %d and PPID %d\n",
               getpid(), getppid());

        printf("My child's PID is %d\n", pid);
    }
    else if (pid == 0) {
        // Child process
        sleep(4);

        printf("I'm the child with PID %d and PPID %d\n",
               getpid(), getppid());

        exit(0);
    }

    printf("PID %d terminates.\n", getpid());

    return 0;
}
```

Task 7: Run the program, observe the output, and explain it.

Zombie processes:

A process that terminates cannot leave the system until its parent accepts its return code. If its parent process is already dead, it'll already have been adopted by the "**init**" process, which always accepts its children's return codes. However, **if a process's parent is alive but never executes a wait ( ), the process's return code will never be accepted and the process will remain a *zombie*.**

Creating a Zombie Process

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid > 0) {
        // Parent process
        printf("I am the parent process. Going into an infinite sleep loop...\n");

        while (1) {
            sleep(100);
        }
    }
    else if (pid == 0) {
        // Child process
        printf("I am the child process. Exiting now...\n");
        exit(0);
    }

    return 0;
}
```

Task 8: Run the above program and understand the output. See the status of both processes using "ps -a". How the child can be terminated?

Multiple Fork() Calls

```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork();   // First fork
    fork();   // Second fork
    fork();   // Third fork

    printf("Hello fork...\n");

    return 0;
}
```

Task-09: Compile and run above program and discuss the output with tree diagram:

Multiple Fork() Using &&

The original uses: `fork() && fork();`

The important point is that `fork()` returns:

- 0 in the child: **false**
- positive PID in the parent: **true**

So the second `fork()` is executed only by the parent from the first `fork()`.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork() && fork();

    printf("UCP\n");

    return 0;

}
```

Task-10: Compile and run above program and discuss the output with tree diagram:

Multiple Fork() Using ||

The original uses: `fork() || fork();`

Since `fork()` returns `0` in the child, the first expression is false in the child, so the second `fork()` executes there. In the parent, the first `fork()` returns a positive PID, so the second `fork()` is skipped.

```
#include <stdio.h>
#include <unistd.h>

int main() {
    fork() || fork();

    printf("UCP\n");

    return 0;
}
```

Task-11: Compile and run above program and discuss the output with tree diagram: